



EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 9

The Unified Software Development Process (USDP / Unified Process)

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 9 of 16

Contents

1. Week 9 Introduction	3
2. Learning Outcomes	3
PART A & B — WHAT IS THE UNIFIED PROCESS?	3
3. Software Development Process	3
4. The Unified Process — and UP vs RUP	4
PART C — ITERATIVE AND INCREMENTAL DEVELOPMENT	5
5. Iterative vs Incremental	5
PART D — THE FOUR MAJOR CHARACTERISTICS	6
6. Use-Case Driven, Architecture-Centred, Iterative & Incremental, Risk-Driven	6
PART E — THE FOUR UP PHASES	8
7. Inception □ Elaboration □ Construction □ Transition	8
8. A Phase Is NOT a Waterfall Stage	9
PART G — PROCESS WORKFLOWS / DISCIPLINES	10
9. Workflows and Their Overlap	10
PART I — ARTEFACTS OF THE UNIFIED PROCESS	11
10. Artefacts	11
PART J — REUSE IN OO DEVELOPMENT	12
11. Reuse — and Its Guidelines	12
PARTS K–M — GENERALISATION, INHERITANCE AND CONCURRENCY	13
12. Generalisation vs Inheritance	13
13. Concurrency	13
14. Connecting UP to the UML Models	14
15. Common Misunderstandings	14
16. Week 9 Summary and Key Terms	15
17. Examination-Style Questions	15
18. References and Further Reading	16

1. Week 9 Introduction

Weeks 4–8 moved from object modelling, through dynamic modelling, to behavioural modelling. Week 9 changes the question again: **“How do we organise the development of the software?”** The distinction is extremely important — **UML tells us how to model a system; the Unified Process tells us how to organise the work of developing that system.**

The **Unified Process (UP)** is an iterative and incremental software-development process strongly associated with object-oriented development and UML. A commercial and particularly well-known implementation is the **Rational Unified Process (RUP)**. The official outline covers the USDP, its aspects and characteristics, phases and iterations, process workflows, artefacts, reuse, concurrency, generalisation and inheritance — all covered this week.

2. Learning Outcomes

By the end of this week, a student should be able to:

- Define the Unified Software Development Process and explain why it suits OO development.
- Explain iterative and incremental development, and distinguish them.
- Explain the major characteristics of UP (use-case driven, architecture-centred, iterative & incremental, risk-driven).
- Distinguish a phase from an iteration, and explain Inception, Elaboration, Construction and Transition.
- Explain milestones, process workflows/disciplines and important artefacts.
- Explain requirements-driven, architecture-centred and risk-driven development.
- Explain reuse, its guidelines, concurrency, generalisation and inheritance.
- Distinguish generalisation from inheritance and connect UP to the UML models already studied.

PART A & B — WHAT IS THE UNIFIED PROCESS?

3. Software Development Process

A **software development process** is an organised set of activities, roles, responsibilities, practices, work products and decisions used to develop and maintain software. In simple language: it tells a development team **what work needs to be done, when, who should do it, and what should be produced**. Without a process, the analyst, programmer, tester and client each build and expect different things — and the manager asks why the project is six months late.

Why Software Needs a Process

Without it, everyone builds something different

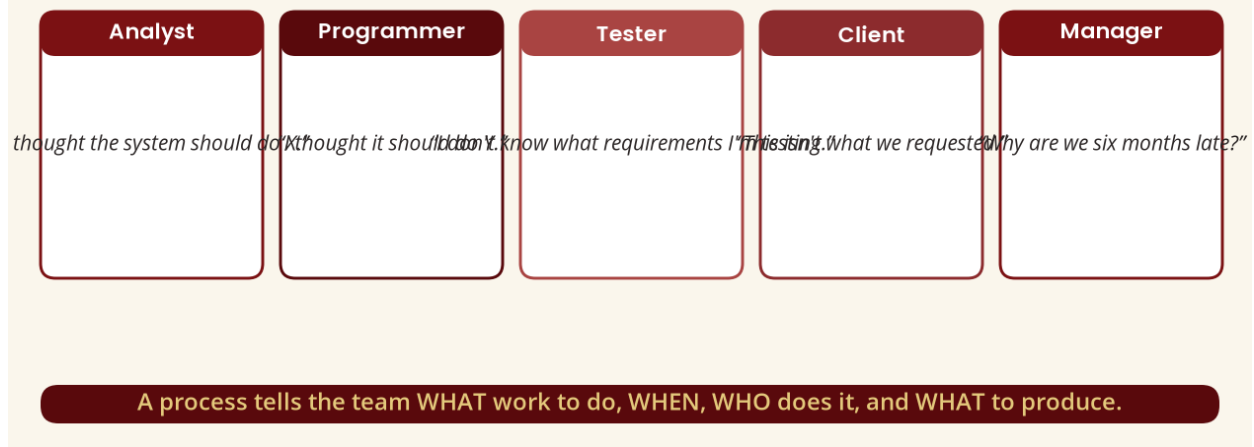


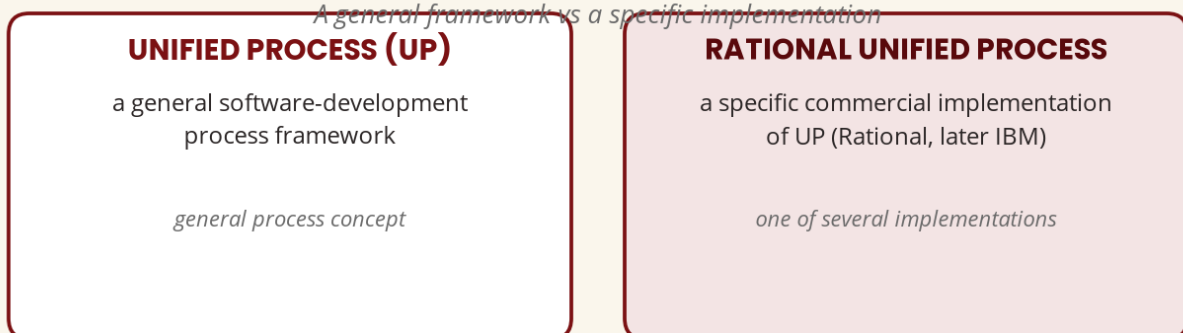
Figure 1 — Why software needs a process

4. The Unified Process — and UP vs RUP

The **Unified Process** is an iterative and incremental software-development process designed around object-oriented development, closely associated with UML and use-case-driven development. The **Rational Unified Process (RUP)** is a specific commercial implementation of UP developed by Rational and later associated with IBM — UP is the general framework; RUP is one implementation of it. A traditional process might treat Requirements □ Analysis □ Design □ Coding □ Testing □ Deployment as separate stages; UP instead encourages repeated cycles with feedback.

UP vs RUP

A general framework vs a specific implementation



UP = general framework · RUP = a specific commercial implementation of UP.

Figure 2 — UP vs RUP

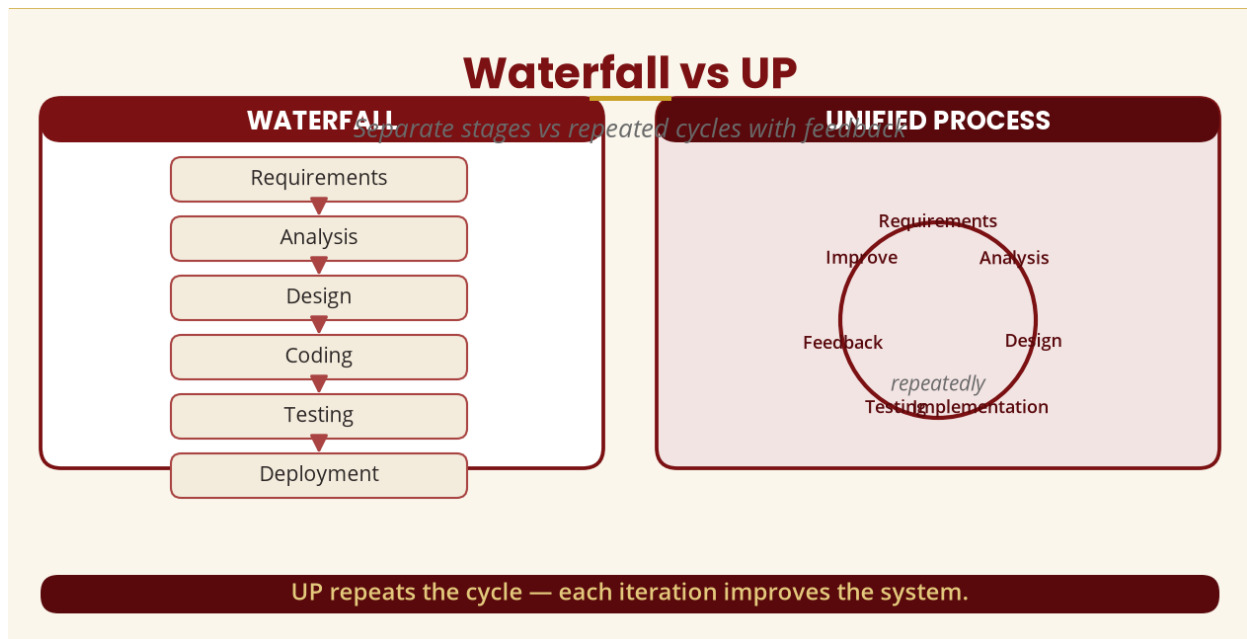


Figure 3 — Waterfall vs UP

PART C — ITERATIVE AND INCREMENTAL DEVELOPMENT

5. Iterative vs Incremental

Iterative means the team repeats development activities in multiple cycles and improves the system from one cycle to the next — like drafting, reviewing and improving an essay. **Incremental** means the system grows by adding usable functionality over time — Increment 1 adds Login, Increment 2 adds course registration, Increment 3 adds assignment submission, Increment 4 adds results. In practice UP uses both: **UP is iterative AND incremental**, with short, timeboxed iterations.

An **iteration** is a planned development cycle in which the team performs some combination of requirements work, analysis, design, implementation, testing and evaluation, ending with a measurable result. We are not waiting until the very end to discover whether the system works.

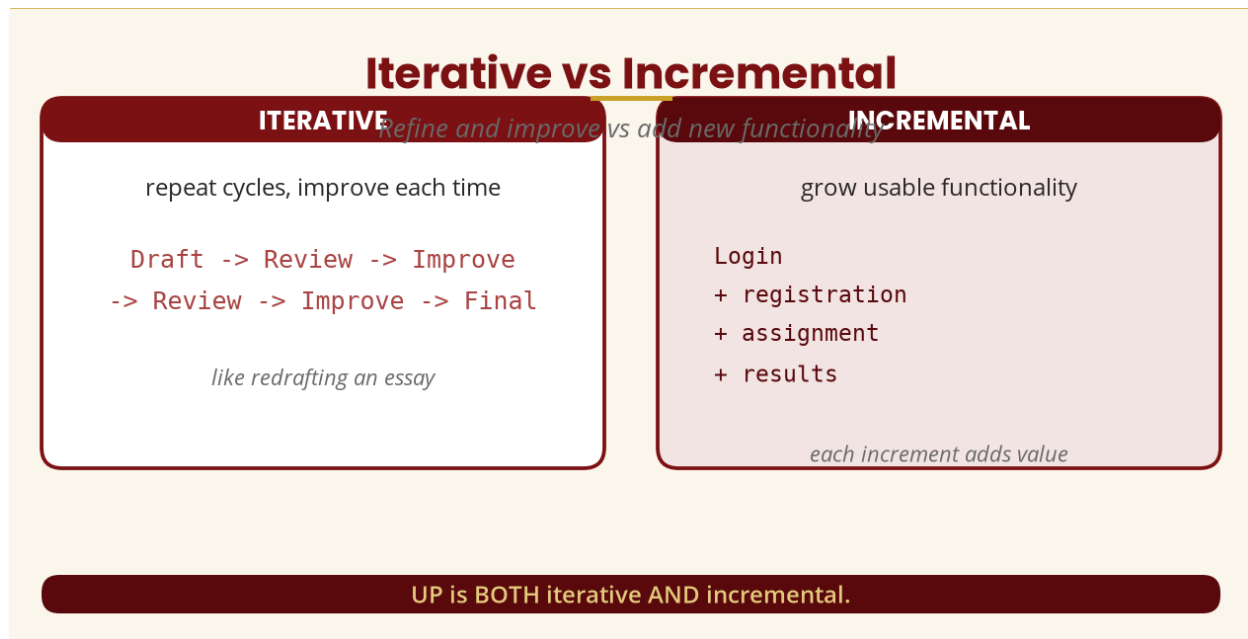


Figure 4 — Iterative vs incremental

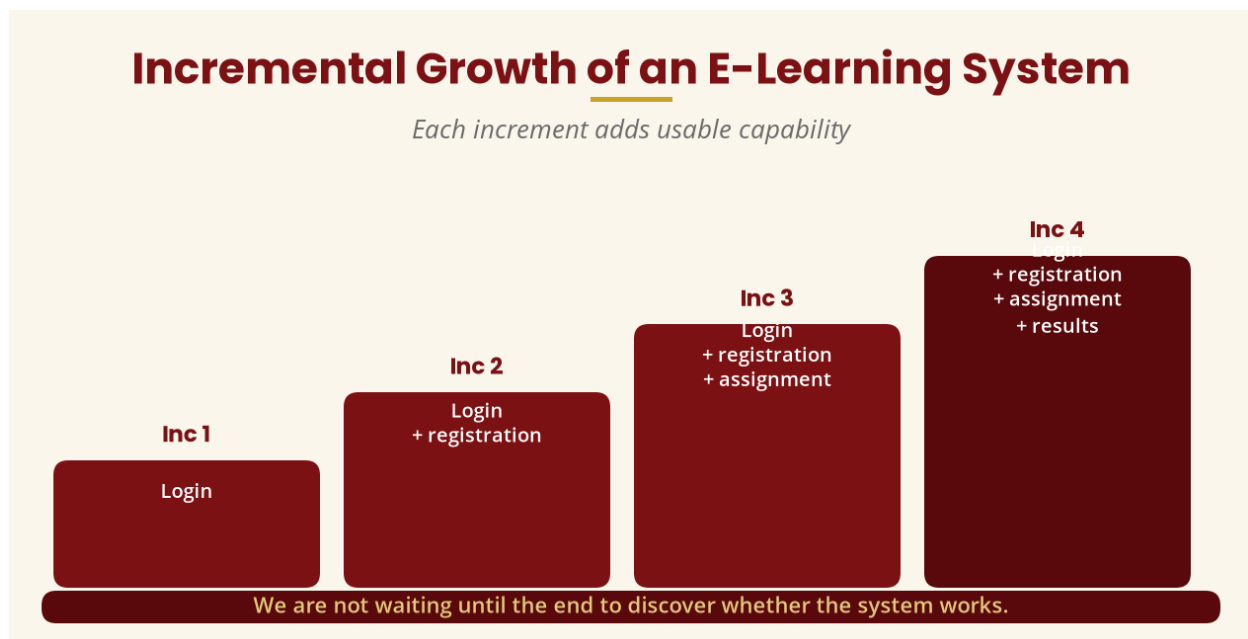


Figure 5 — Incremental growth of an e-learning system

PART D — THE FOUR MAJOR CHARACTERISTICS

6. Use-Case Driven, Architecture-Centred, Iterative & Incremental, Risk-Driven

Use-case driven: development is organised around use cases. UC-01 Register Course becomes a thread running through requirements, analysis, design, implementation and testing — identifying the objects (Student, Course, Registration), the classes (RegistrationController, CourseService), the code and the test scenarios.

Architecture-centred: UP establishes and validates a strong architectural foundation early, especially during Elaboration. Software architecture is the high-level organisation of a system (User Interface → Business Logic → Data Access → Database). A system built for 100 students may collapse with 100,000, so major architectural risks are identified early.

Iterative and incremental: the system is developed through repeated cycles, each improving or extending it. **Risk-driven:** UP identifies and addresses significant risks early rather than “building the easiest things first” — a mobile banking system that must process 5,000 transactions per second is tested early, not left to the end.

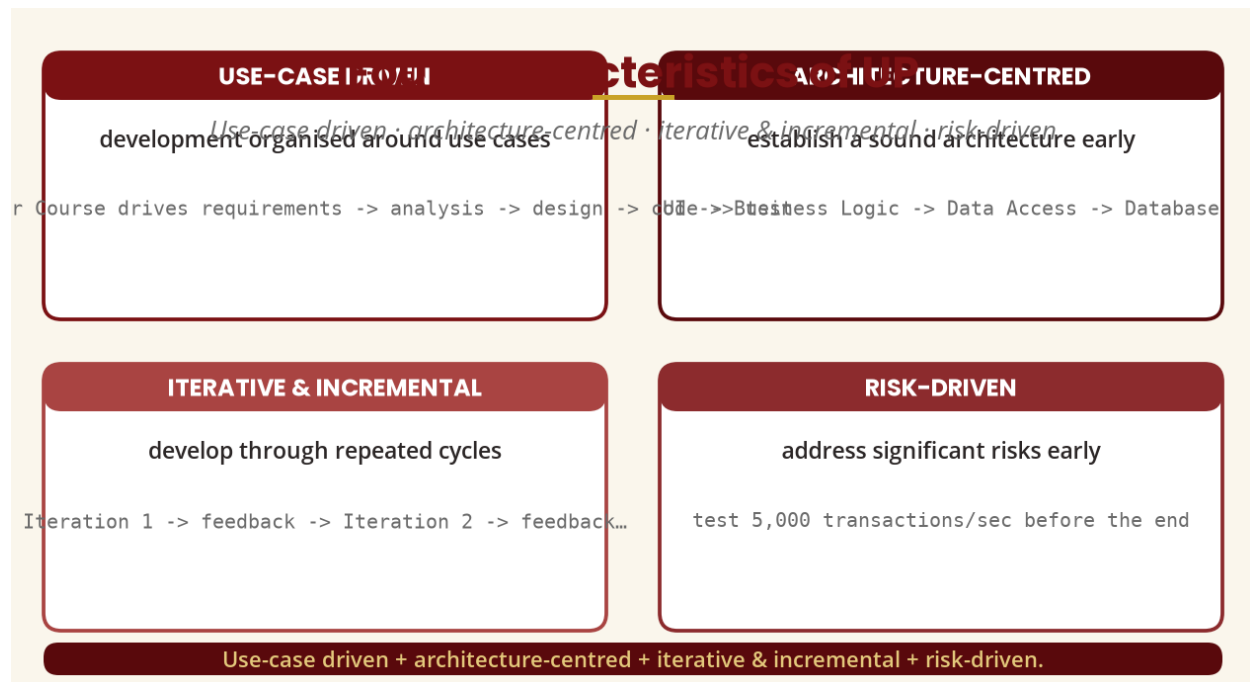


Figure 6 — The four characteristics

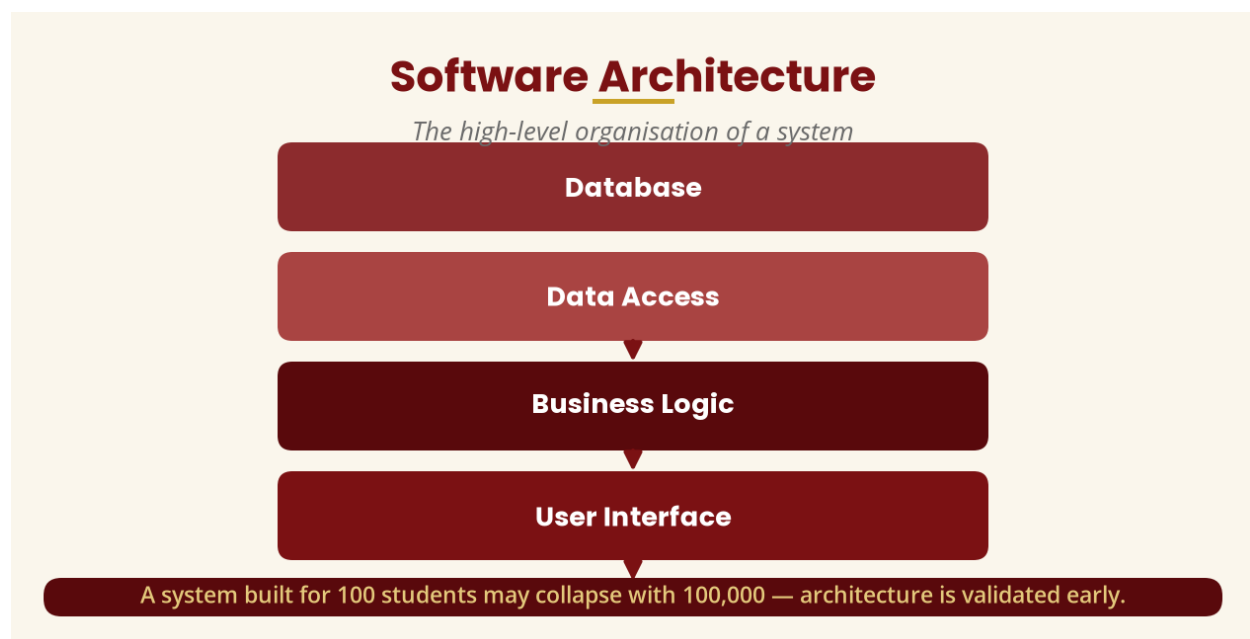


Figure 7 — Software architecture layers

PART E — THE FOUR UP PHASES

7. Inception Elaboration Construction Transition

Inception asks “should we build this system, and what are we trying to build?” — it establishes the business case, scope, high-level requirements, major actors and use cases, cost, schedule, risks and success criteria, ending at the **Lifecycle Objectives Milestone** (a go/no-go decision).

Elaboration asks “can we build it, and with what architecture?” — refining requirements, analysing the domain, establishing architecture and addressing major technical risks (often via an executable architectural prototype), ending at the **Lifecycle Architecture Milestone**.

Construction asks “can we build the complete system?” — most detailed implementation takes place here, through iterations that add course registration, payments, results, reports and administration.

Transition asks “can real users use it in the real environment?” — deployment, installation, data migration, training, beta testing, defect fixing and final adjustments before release.

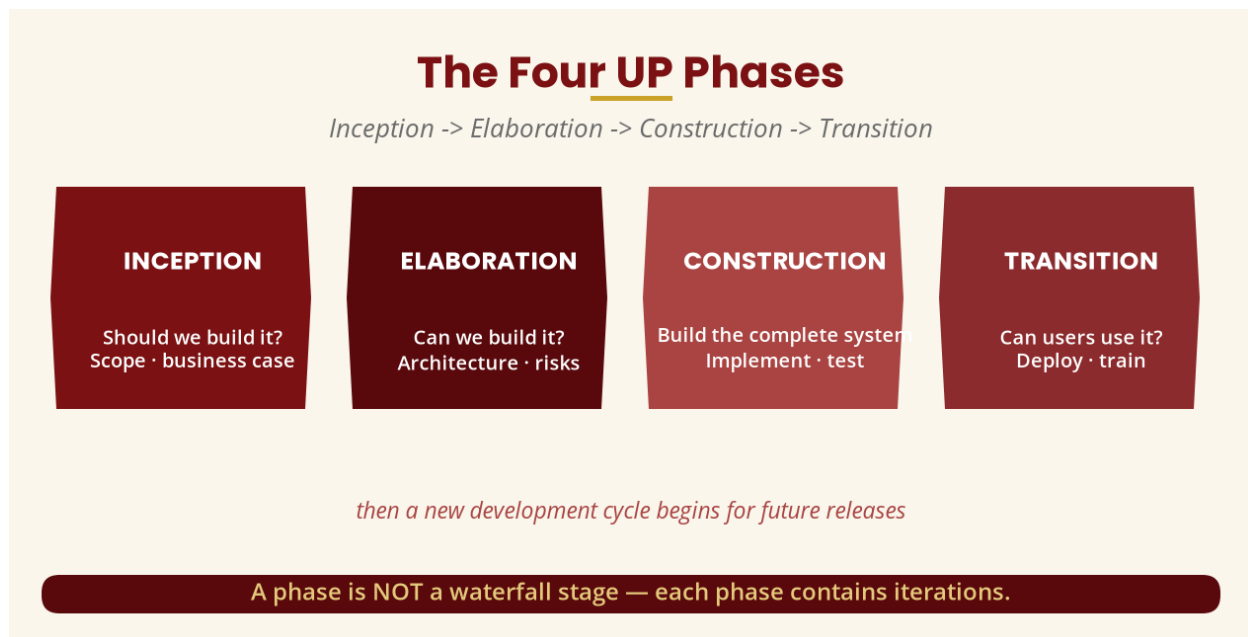


Figure 8 — The four phases

The Four Phases Compared

Each phase has its own question and focus

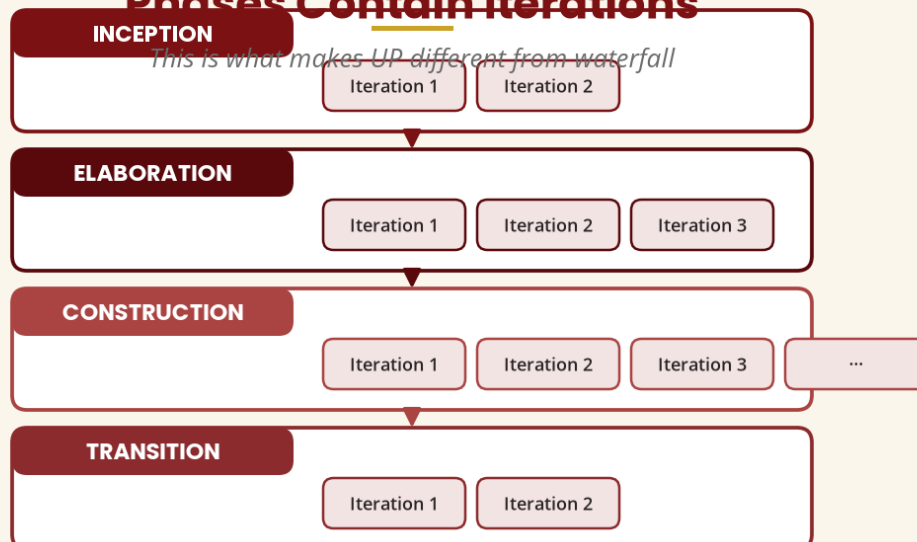
Inception	Should we build it?	Scope, business case, major requirements & goals
Elaboration	Can we build it?	Architecture, detailed requirements, major risks
Construction	Can we build the complete product?	Implementation, integration, testing
Transition	Can users successfully use it?	Deployment, training, acceptance, fixes

Figure 9 — The four phases compared

8. A Phase Is NOT a Waterfall Stage

Students may look at Inception → Elaboration → Construction → Transition and think it is “waterfall with different names”. No. **A phase contains one or more iterations** — iterations provide planned intervals that deliver incremental value within each phase. That is what makes UP fundamentally different from treating the four phases as four rigid sequential stages.

Phases Contain Iterations



Iterations provide planned intervals that deliver incremental value within each phase.

Figure 10 — Phases contain iterations

PART G — PROCESS WORKFLOWS / DISCIPLINES

9. Workflows and Their Overlap

A **workflow** (discipline) is a set of related activities performed to achieve a development objective. Core technical disciplines are **Business Modeling** (how does the organisation operate?), **Requirements** (what must the system do?), **Analysis & Design** (how should it be structured — this is where our UML models matter most), **Implementation**, **Test** and **Deployment**. Supporting disciplines are **Configuration & Change Management**, **Project Management** and **Environment**.

A common misconception is “first requirements, then design, then implementation, then testing”. In UP the disciplines overlap: during an iteration requirements → analysis → design → implementation → testing → feedback → refined requirements, and the **intensity** of each discipline changes across phases — requirements are heavy in Inception and Elaboration, coding is heavy in Construction, testing and deployment are heavy in Transition. Coding can even occur in Elaboration (architectural prototypes).

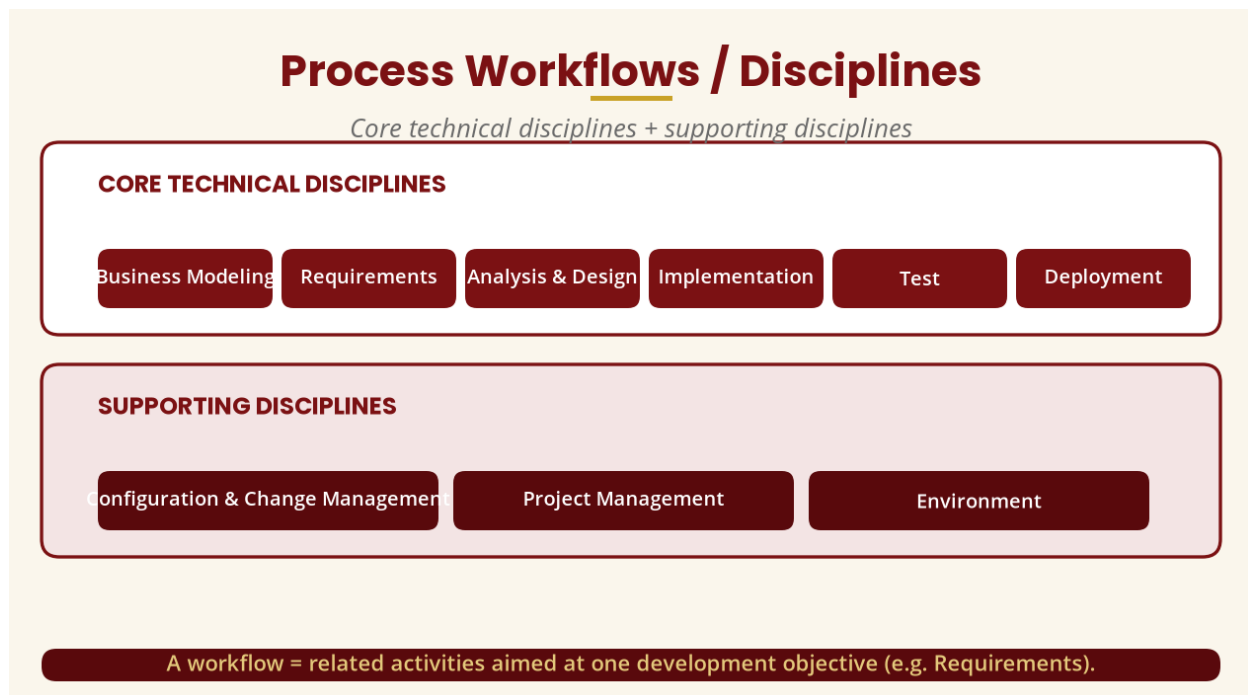


Figure 11 — The core and supporting workflows

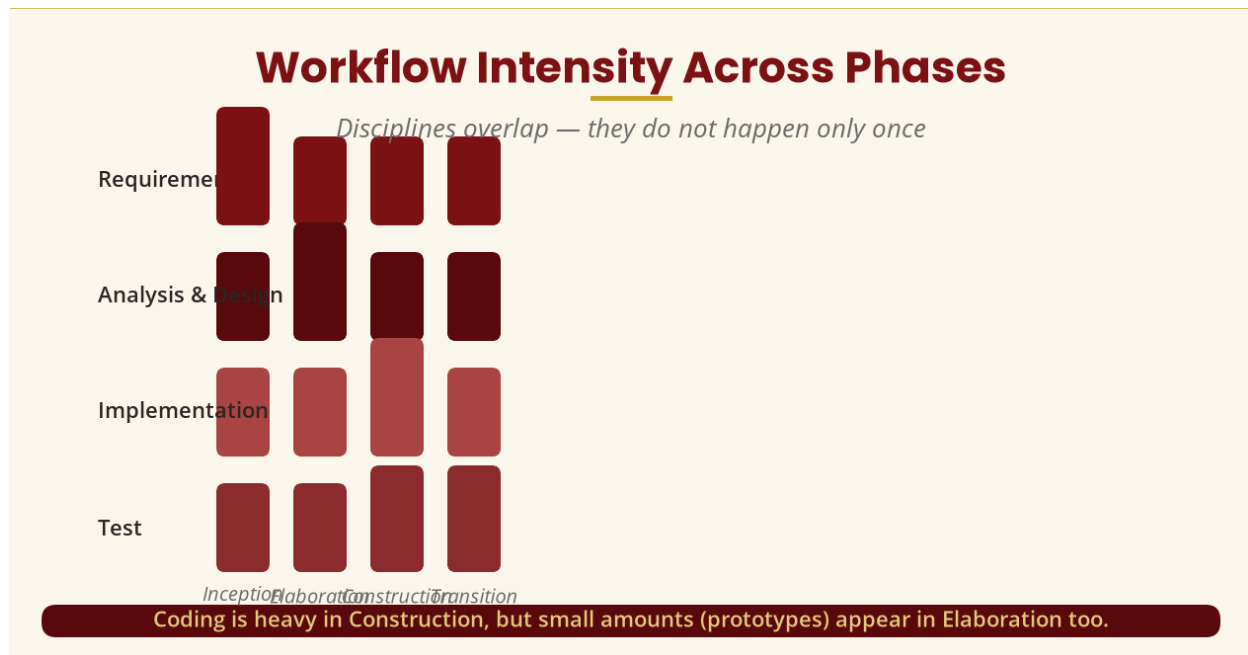


Figure 12 — Workflow intensity across phases

PART I — ARTEFACTS OF THE UNIFIED PROCESS

10. Artefacts

An **artefact** is a work product created, modified or used during development — evidence of the team's work. Examples: the Vision document, requirements specification, use-case model, risk list, project plan, glossary, class and sequence diagrams, architecture document, source code, test plan and cases, deployment package and user manual. Across the phases: Inception produces the Vision, Business Case, initial use-case model, risk assessment and project plan; Elaboration produces detailed requirements, the architecture document, the domain model and an architectural prototype; Construction produces source code, components, test cases and working builds; Transition produces the deployment package, user documentation and release notes.

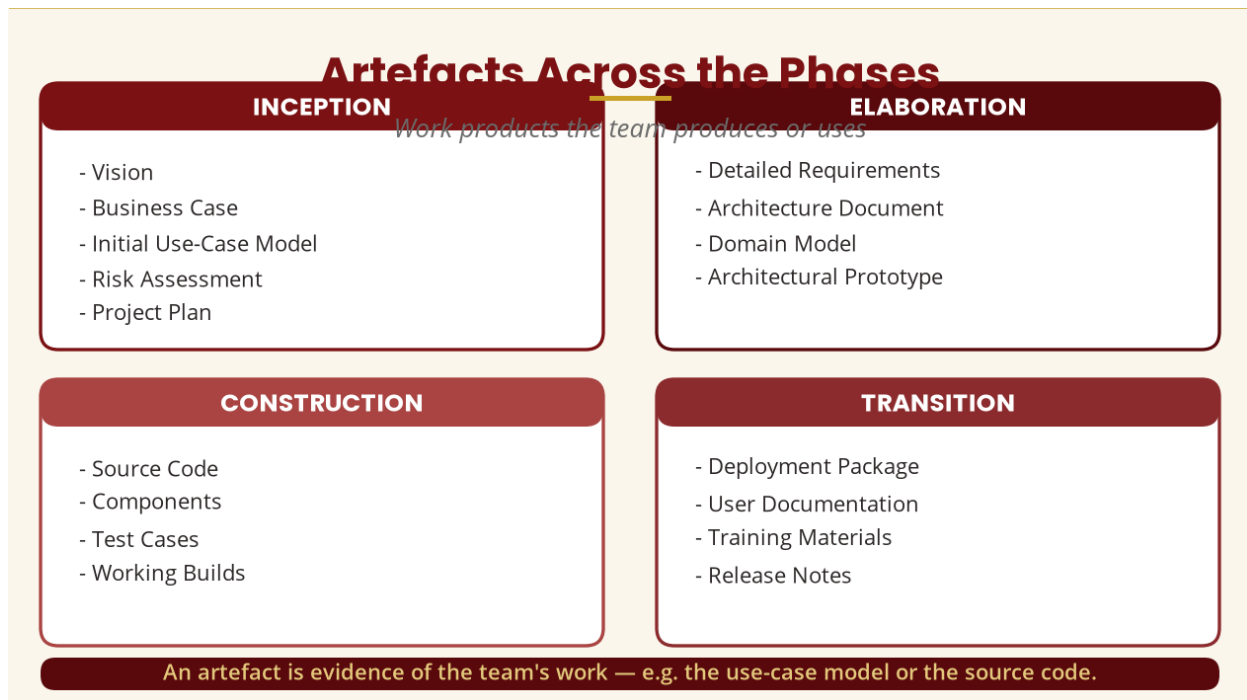


Figure 13 — Artefacts across the phases

PART J — REUSE IN OO DEVELOPMENT

11. Reuse — and Its Guidelines

Software reuse means using an existing asset again instead of creating the same functionality from scratch — classes, components, libraries, frameworks, services, APIs, design patterns and architectural components. Reuse can reduce time and cost, improve consistency and leverage tested components. But reuse is not automatically good: before adopting an asset, evaluate its **functional suitability, quality, security, compatibility, maintainability, licence, performance and cost**.

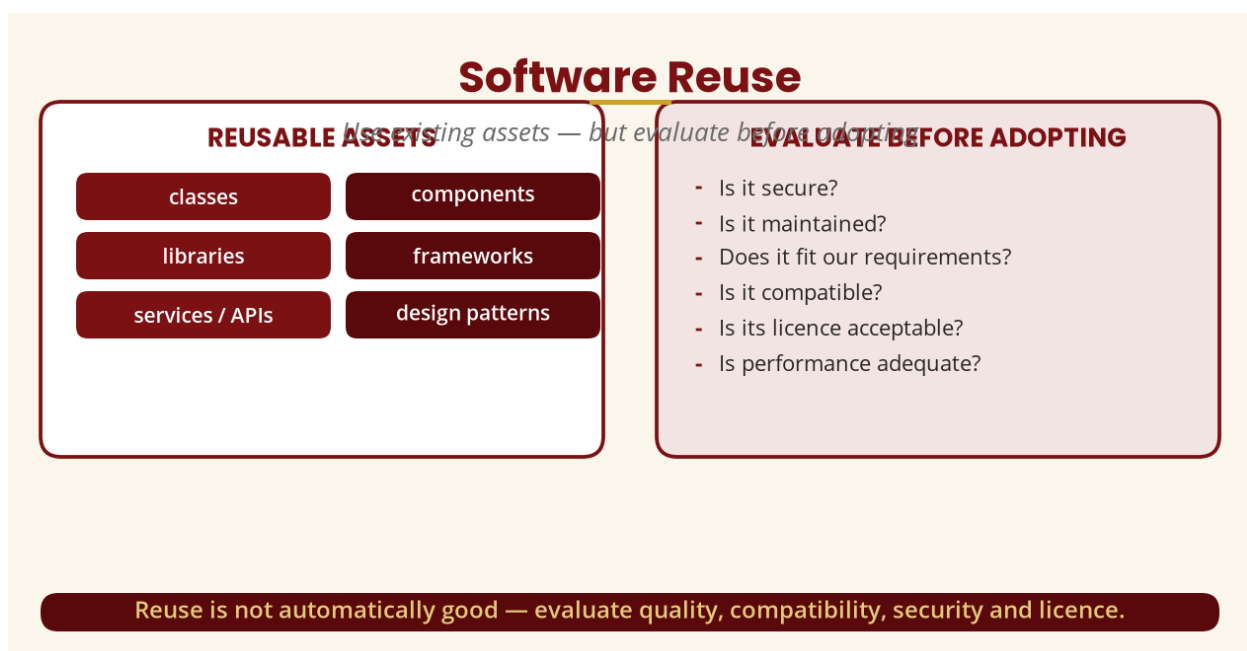


Figure 14 — Reuse: assets and evaluation

PARTS K–M — GENERALISATION, INHERITANCE AND CONCURRENCY

12. Generalisation vs Inheritance

Generalisation is a modelling/design relationship in which a more general concept represents characteristics shared by more specialised concepts (User generalises Student and Lecturer; Student IS-A User). **Inheritance** is the programming-language mechanism through which a specialised class acquires accessible attributes and operations from a more general class (class Student extends User). They are closely related but not identical: generalisation is the conceptual relationship; inheritance is the implementation mechanism. The **is-a test** — “Is Student a User?” yes; “Student has a Course” is a different relationship.

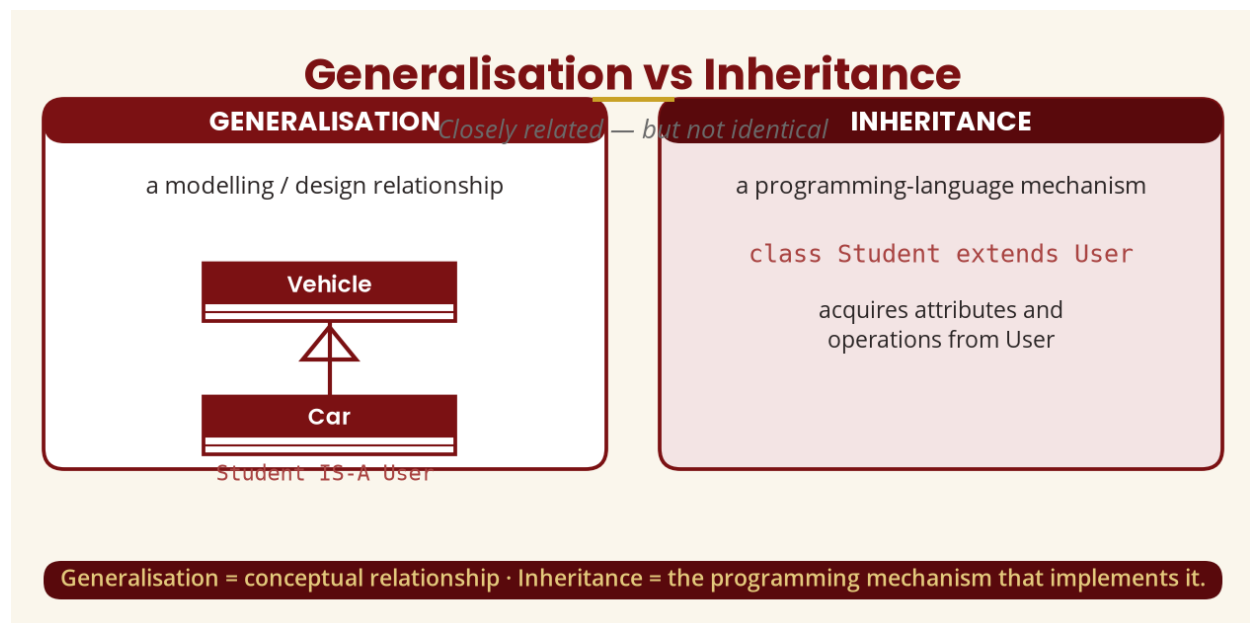


Figure 15 — Generalisation vs inheritance

13. Concurrency

Concurrency means multiple activities or operations can make progress during overlapping periods. After a payment succeeds, updating inventory, generating a receipt, notifying the warehouse and sending an email can happen independently. Concurrency affects architecture, performance, responsiveness, thread/process design, synchronisation and data consistency.

A classic OOAD problem: a course has capacity 1 and two students register at exactly the same time. Both systems check “available seats = 1”, both register successfully — now registered = 2 while capacity = 1. **Concurrency must sometimes be addressed at the analysis/design level.**

Concurrency — a Real Problem

Two students, one remaining seat

Course capacity = 1 (the last seat)

Student A -> Register

Student B -> Register

Both systems check: Available seats = 1

Student A -> SUCCESS

Student B -> SUCCESS

Registered = 2 · Capacity = 1

Without synchronisation this is a serious system error — concurrency matters at analysis/design level.

Figure 16 — The concurrency problem

14. Connecting UP to the UML Models

The class/object, dynamic and behavioural models of previous weeks do not exist in isolation — within UP they become **artefacts** used during development: requirement □ use case □ use-case diagram □ activity diagram □ sequence diagram □ class/object model □ design □ implementation. **The Unified Process provides the development process; UML provides many of the modelling languages used within that process.**

Worked example — the Mzuni Online Student Registration System: **Inception** identifies actors (Student, Registrar, Finance Officer, Payment Gateway), use cases (Login, Register Course, Drop Course, Pay Fees, View Registration, Generate Reports) and risks (security, payment integration, database performance, migration, adoption). **Elaboration** investigates authentication, database and payment architecture and builds a prototype. **Construction** delivers iterations — login, catalogue, registration, payments, reports. **Transition** installs, migrates records, trains administrators, pilots, fixes defects and launches.

15. Common Misunderstandings

- **“UP is the same as Waterfall.”** — No: phases contain iterations and activities overlap.
- **“Inception means collecting every requirement.”** — Inception establishes the business case, scope and major requirements; requirements are refined over iterations.
- **“Coding only happens in Construction.”** — Coding can occur earlier (architectural prototypes in Elaboration).
- **“Testing starts after all coding is finished.”** — Testing occurs throughout iterative development.
- **“Generalisation and inheritance are exactly the same.”** — Closely related, but generalisation is the modelling relationship and inheritance is the programming mechanism.

- **“Reuse means copy and paste code.”** — Reuse spans classes, libraries, services, frameworks, APIs and patterns, and requires evaluation.

16. Week 9 Summary and Key Terms

Term	Simple meaning
Unified Process	Organised OO software-development process.
RUP	Rational's commercial implementation of UP.
Iterative	Repeat and improve across cycles.
Incremental	Add functionality progressively.
Use-case driven	Use cases guide development.
Architecture-centred	Build and validate important architecture early.
Risk-driven	Address significant risks early.
Inception / Elaboration	Establish business case & scope / refine requirements & architecture.
Construction / Transition	Build the system / deploy and move to users.
Phase / iteration / workflow / artefact	Major lifecycle period / cycle within a phase / related activities / work product.
Reuse	Use existing software assets (with evaluation).
Generalisation / inheritance	Modelling relationship / programming mechanism.
Concurrency	Overlapping execution of activities.

THE BIG PICTURE — FIVE THINGS TO REMEMBER

1. UP is **iterative and incremental**. 2. UP is **use-case driven**. 3. UP is **architecture-centred**. 4. UP has **four phases** — Inception □ Elaboration □ Construction □ Transition. 5. **Phases contain iterations** — that is what makes UP different from four rigid sequential stages. Week 8 asked “what should the system do?”; Week 9 asks “how should the team organise the work?”; Weeks 10+ ask “how should we design the software?”

17. Examination–Style Questions

Essay (25 marks). Discuss the Unified Software Development Process and explain how its four phases contribute to successful object-oriented software development — covering the definition, iterative and incremental nature, use-case driven development, architecture, risk management, the four phases, milestones and artefacts.

Scenario. A hospital intends to develop an online patient management system to register patients, book appointments, process payments, store medical records and generate reports, and is particularly concerned about security, performance and integration with existing systems. Required: explain how UP could be used; what should happen during Inception; why Elaboration is particularly important; give three architectural risks; suggest three artefacts; explain how iterations could be used during Construction; and explain the activities of Transition.

18. References and Further Reading

Primary / authoritative

- Object Management Group. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. OMG — the formal specification, adopted December 2017.
- IBM. *Project Planning — Phases and Iterations* (IBM Rational documentation; identifies the four RUP phases and iterations).
- IBM Rational. *Rational Unified Process: Best Practices for Software Development Teams*. IBM.
- Project Management Institute. *Project Management and the Rational Unified Process for Software Development Projects*. PMI.
- The Open University. *Approaches to Software Development — Introducing the Unified Process*.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling*.
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML*.
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide*.
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design*.
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements*.
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML*.